Lenguajes Formales, Autómatas y Computabilidad

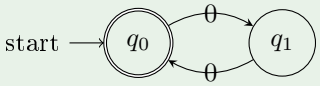
# Práctica 2: Autómatas finitos

27 de septiembre de 2026

**Ejercicio 1.** Construir autómatas finitos para los siguientes lenguajes:

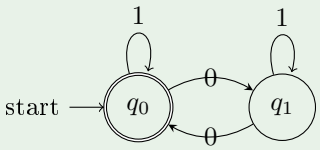
a) Cadenas sobre $\Sigma = \{0\}$ de longitud par.

**Sol:** DFA



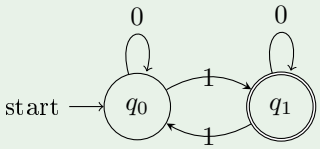

b) Cadenas sobre $\Sigma = \{0, 1\}$ con cantidad par de ceros.

**Sol:** DFA



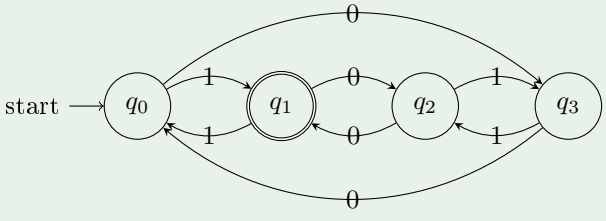

c) Cadenas sobre $\Sigma = \{0, 1\}$ con cantidad impar de unos.

**Sol:** DFA




d) Cadenas sobre $\Sigma = \{0, 1\}$ con cantidad par de ceros y cantidad impar de unos.

**Sol:** DFA (es la interseccion de b y c)



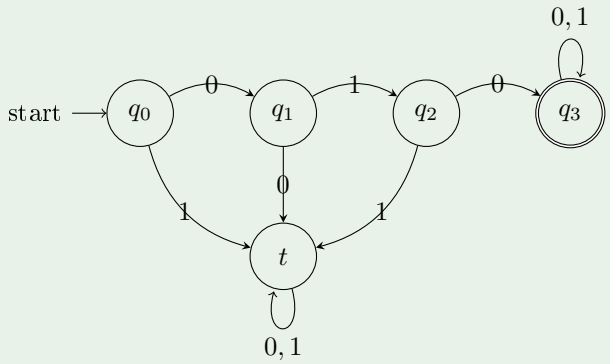

e) Cadenas sobre $\Sigma = \{0, 1\}$ que, interpretadas como un número binario, sean congruentesa cero módulo 5.

**Sol:** TO DO

**Ejercicio 2.** Construir autómatas finitos para los siguientes lenguajes sobre $\Sigma = \{0, 1\}$:

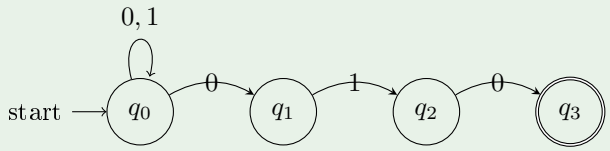
a) Cadenas que comiencen con 010.

**Sol:** DFA



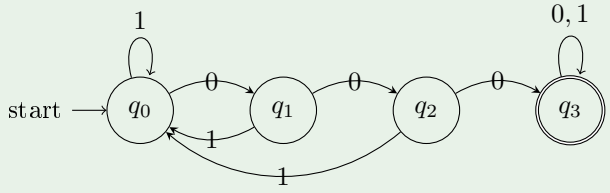

b) Cadenas que terminen con 010.

**Sol:** NFA



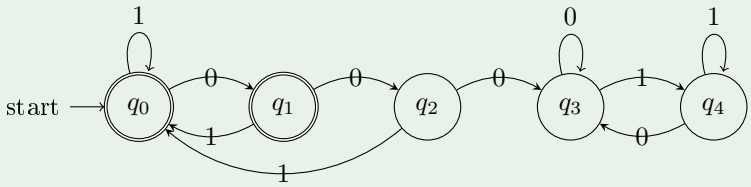

c) Cadenas que contengan la subcadena 000.

**Sol:** DFA



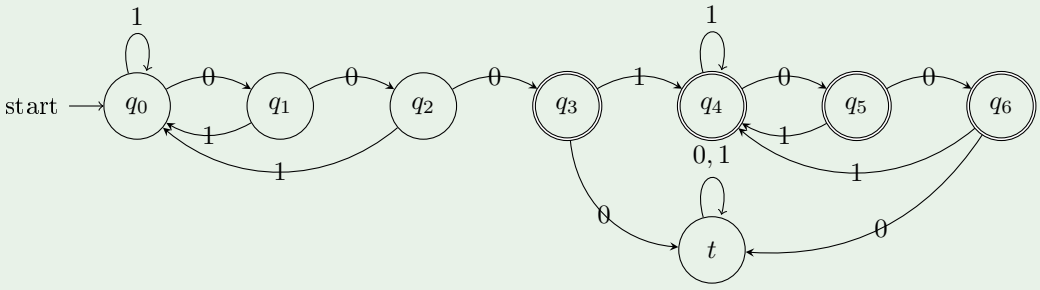

d) Cadenas que no contengan la subcadena 000.

**Sol:** DFA (es el complemento del anterior)



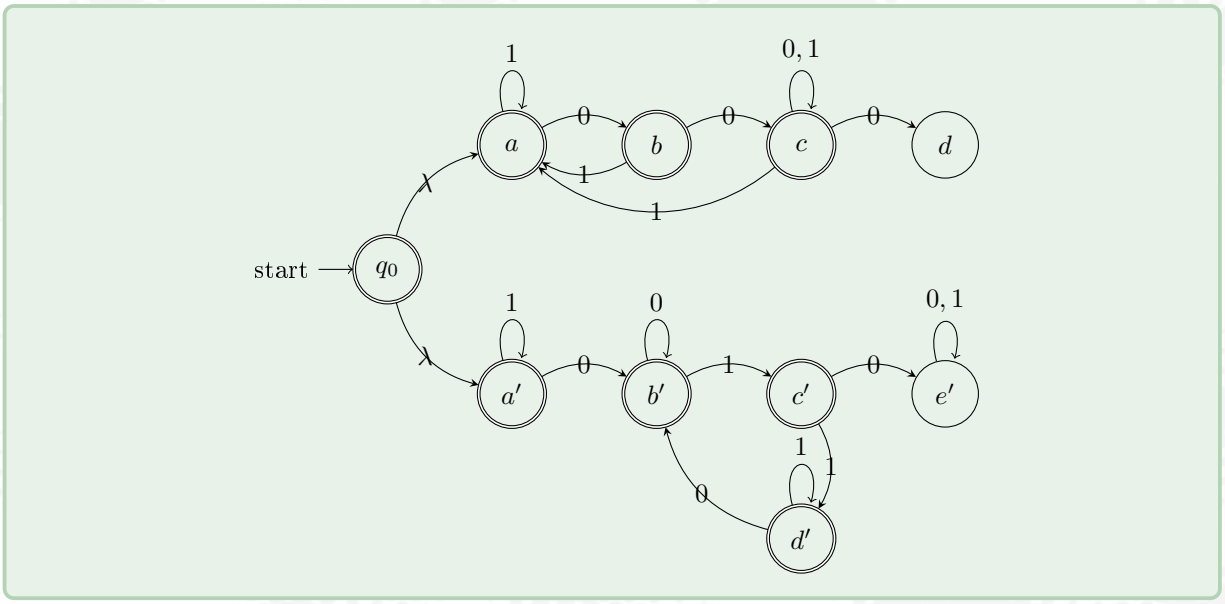

e) Cadenas que contengan la subcadena 000 exactamente una vez
(la cadena 0000 no pertenece a este lenguaje).

**Sol:** DFA

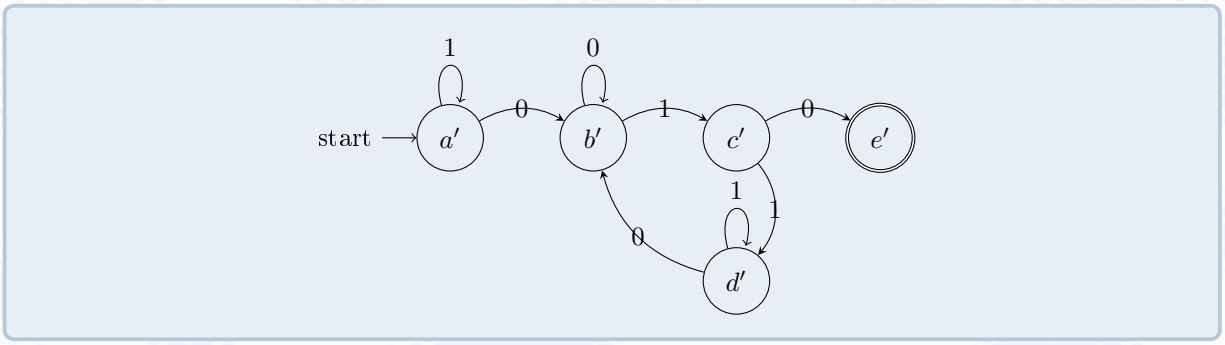


f) Cadenas que no contengan la subcadena 000 ni la 010.

**Sol:** AFD-$\lambda$ (es el complemento de la union del que acepta 000 y 010)
(para sacar la union hago un nuevo estado inicial que apunte a los previos con una transicion $\lambda$, para sacar el complemento, los que eran finales pasan a ser no finales y viceversa)




En azul, el DFA que acepta las cadenas con 010 en cualquier lado:




**Ejercicio 3.** TO DO

**Ejercicio 4.** Dado un autómata finito para $\mathcal{L}$, indicar cómo construir autómatas finitos para los siguientes lenguajes. Indicar en cada caso si es necesario que el autómata de entrada sea determinístico o no, y de qué tipo es el autómata resultante.

a) $\mathcal{L}^c$, el complemento de $\mathcal{L}$.
**Sol:** Los que eran estados finales pasan a ser estados no finales y viceversa. El automata anterior tiene que ser un AFD completo. El automata resultante sera un AFD completo

b) $\mathcal{L}^*$, la clausura de Kleene de $\mathcal{L}$.
**Sol:** Agregando transiciones $\lambda$ desde todos los estados finales hacia estado inicial original (y haciendo que el estado inicial acepte la cadena vacia tambien). El automata anterior podia ser de cualquier tipo. El automata resultante sera un AFND-$\lambda$

c) $\mathcal{L}^r$, la reversa de $\mathcal{L}$.
**Sol:** Se invierten todas las flechas y los estados finales pasan a ser iniciales (apuntados por un nuevo inicial con saltos $\lambda$) mientras que el estado inicial pasa a ser final. El automata anterior podia ser de cualquier tipo. El automata resultante sera un AFND-$\lambda$

d) $\text{Ini}(\mathcal{L}) = \{\alpha \mid \exists \beta \text{ tal que } \alpha\beta \in \mathcal{L}\}$, los prefijos de $\mathcal{L}$.
**Sol:** Marcando todos los estados como finales. El automata anterior tiene que ser AFD o AFND sin estados inutiles. Si era antes un AFD, el resultante tambien

e) $\text{Fin}(\mathcal{L}) = \{\alpha \mid \exists \gamma \text{ tal que } \gamma\alpha \ \ \mathcal{L}$, los sufijos de $\mathcal{L}$.
**Sol:** Marcando todos los estados como iniciales (tambien mediante un nuevo estado inicial que apunte a ellos mediante transiciones $\lambda$). El automata anterior podia ser de cualquier tipo. El automata resultante sera un AFND-$\lambda$

f) $\text{Sub}(\mathcal{L}) = \{\alpha \mid \exists(\beta, \gamma)$ tales que $\gamma\alpha\beta \in \mathcal{L}\}$, las subcadenas de $\mathcal{L}$.
**Sol:** Agregando un nuevo estado inicial que apunte con transiciones $\lambda$ a todos los estados que eran alcanzables desde el anterior estado inicial, ahora pasan a ser finales a todos los estados desde los que se podia llegar a un estado final antes. El automata anterior debe ser un AFD o un AFND sin estados inutiles/trampa. El automata resultante sera un AFND-$\lambda$

g) $\text{Máx}(\mathcal{L}) = \{\alpha \in \mathcal{L} \mid \forall\omega \in \Sigma^+, \alpha\omega \notin \mathcal{L}\}$, las cadenas maximales de $\mathcal{L}$.
**Sol:** Sacando los estados finales de antes que apuntaban a otro estado final (o que eventualmente llegarian a otro estado final). El automata anterior debe ser AFD y completo. El resultante tambien sera AFD

h) $\text{Mín}(\mathcal{L}) = \{\alpha \in \mathcal{L} \mid$ ningún prefijo propio de $\alpha$ pertenece a $\mathcal{L}\}$, las cadenas minimales de $\mathcal{L}$.
Es decir, $\text{Mín}(\mathcal{L}) = \{\alpha \in \mathcal{L} \mid \nexists(\omega_1, \omega_2)$ tales que $\alpha = \omega_1\omega_2 \wedge \omega_1 \in \mathcal{L} \wedge \omega_2 \neq \lambda\}$.
**Sol:** Sacando los estados finales de antes que estaban luego de otro final. El automata anterior debe ser AFD y completo. El resultante tambien sera AFD

**Ejercicio 5.** Dados autómatas finitos para $\mathcal{L}_1$ y $\mathcal{L}_2$ indicar cómo construir autómatas finitos para los siguientes lenguajes, con las mismas consideraciones que en el ejercicio anterior:

a) $\mathcal{L}_1 \cup \mathcal{L}_2$
**Sol:** O los dibujas otra vez y haces un nuevo estado inicial con transiciones $\lambda$ a los anteriores iniciales o vas haciendo nuevos estados de la forma (automata1, automata2) donde la primera parte de la tupla es a la que llegabas desde el primer automata y la segunda desde el segundo, al final los estados finales van a ser los que tengan alguna parte de la tupla que antes era final. Para la primera forma podian ser cualquier automata y devuelve un AFND-$\lambda$, para la segunda si o si es necesario que sean AFD

b) $\mathcal{L}_1 \cap \mathcal{L}_2$
**Sol:** Lo mismo que en la union pero los estados finales seran los que tengan las dos partes de la tupla como finales. Los automatas anteriores deben ser AFD

c) $\mathcal{L}_1.\mathcal{L}_2$
**Sol:** (Concatenacion) Se conectan los estados finales de $\mathcal{L}_1$ (mediante saltos $\lambda$) con el inicial de $\mathcal{L}_2$ (y dejan de ser finales si $\mathcal{L}_2$ no aceptaba la cadena vacia). Los automatas anteriores pueden ser de cualquier tipo y devuelve un AFND-$\lambda$

d) $\mathcal{L}_1 \backslash \mathcal{L}_2$
**Sol:** es lo mismo que $\mathcal{L}_1 \cup \mathcal{L}_2^c$. Antes tenian que ser AFD completos y devuelven un AFD completo

**Ejercicio 6.** TO DO

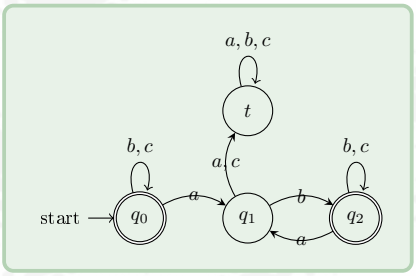
**Ejercicio 7.** Dar un autómata finito determinístico que acepte todas las cadenas sobre el alfabeto $\{a, b, c\}$ que cumplan simultáneamente las siguientes reglas:

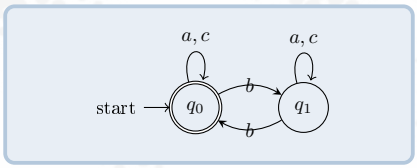
a) Cada $a$ debe estar seguida inmediatamente de una $b$.

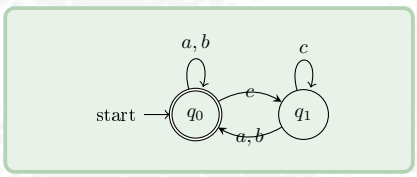
b) La cantidad de $b$ debe ser par.

c) La cadena no debe terminar en $c$.

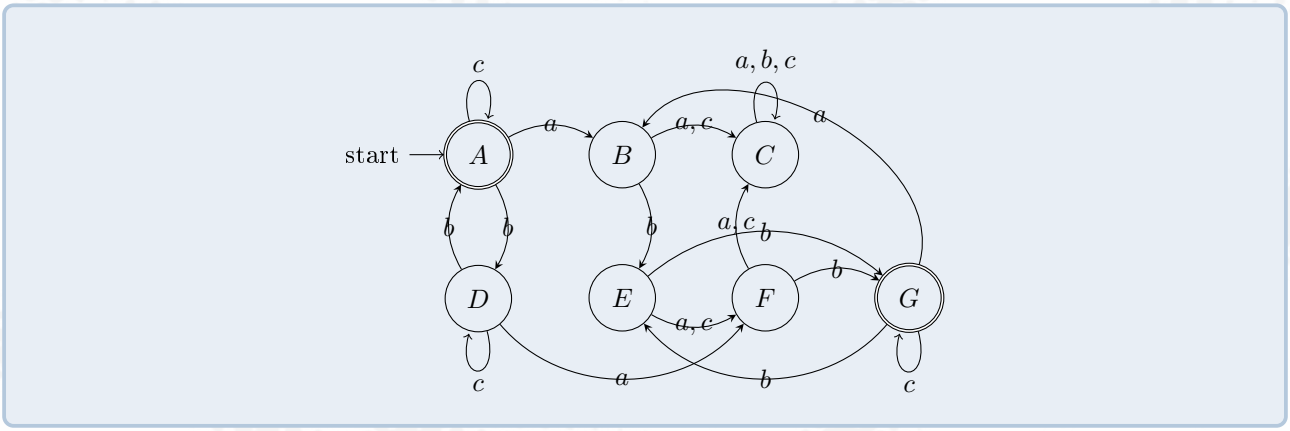
**Sol:** Hago los 3 incisos por separado y luego los uno mediante la union (:p)



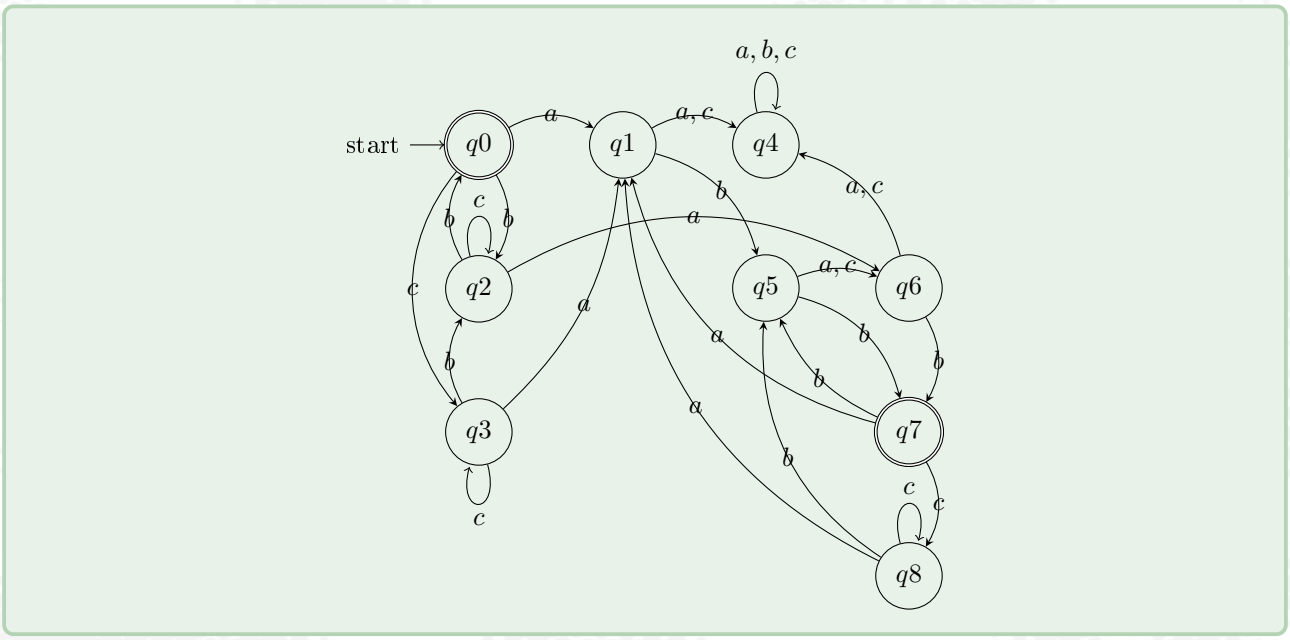




$\mathcal{L}_1 \cap \mathcal{L}_2$**:**




$\mathcal{L}_1 \cap \mathcal{L}_2 \cap \mathcal{L}_3$ :




**Ejercicio 8.** Decimos que una subcadena de otra cadena es un grupo de repetición (o meseta) si todos sus símbolos son iguales y ninguno de los símbolos adyacentes a ella coincide con los que la forman. Por ejemplo, en la palabra aaabbbbaaa hay tres grupos de repetición (aaa, bb bb y aaa).
Se considera el lenguaje $\mathcal{L}$ sobre el alfabeto $\{a, b\}$ formado por las cadenas en las que, si existen grupos de repetición, su longitud es alternativamente par e impar. Es decir, la palabra aabbbaaaab pertenece al lenguaje $\mathcal{L}$, ya que esta formada por cuatro grupos de repetición de longitudes 2, 3, 4 y 1, mientras que la palabra bbaa no pertenece, al estar formada por dos grupos de repetición de longitudes 2 y 2.
Dar un autómata finito que acepte $\mathcal{L}$.

**Sol:** CREO

